Scheduly — Single Agent 기반 개인 비서 시스템

이메일 분류 · 채용공고 추출 · 이력서 매칭 · 캘린더 관리를 하나의 AI 에이전트로 처리하는 통합 어시스턴트

목차

1. 문제 정의
2. 해결 방향
3. 트레이드오프 및 설계 결정
4. 시스템 아키텍처
5. 에이전트 파이프라인
6. 프로젝트 진화 경로
7. 기술 스택
8. 실행 방법

---

1. 문제 정의

배경

구직 활동 중인 사용자는 매일 수십 통의 이메일을 받는다. 채용 플랫폼 알림, 기업 인사팀 연락, 뉴스레터, 업무 메일이 한데 뒤섞여 있어 중요한 채용 정보를 놓치거나, 일정 관리에 실패하는 경우가 빈번하다.

핵심 문제

문제	설명
이메일 과부하	Gmail/Daum 이메일이 카테고리 없이 섞여 있어 채용 관련 메일을 수동으로 분류해야 함
정보 파편화	채용공고 URL, 마감일, 회사 정보가 이메일 본문에 흩어져 있어 일괄 관리 불가
이력서-공고 매칭 부재	보유 이력서와 수신한 채용공고 사이의 적합도를 빠르게 파악하는 수단이 없음
캘린더 분산 관리	면접 일정, 서류 마감일을 이메일과 별도로 캘린더에 수동 입력해야 함
컨텍스트 단절	위 작업들이 서로 다른 앱에 분산되어 있어 흐름이 끊김

---

2. 해결 방향

단일 에이전트(Single Agent) 접근

여러 도메인에 걸친 작업을 **하나의 LLM 에이전트**가 도구(Tool)를 선택·실행하는 ReAct 루프로 처리한다. 사용자는 자연어로 명령하고, 에이전트가 필요한 도구를 자율적으로 조합한다.

```
사용자: "이메일 분류하고 채용공고 추출해서 5월 3일 면접 일정도 등록해줘"
  ↓
에이전트: fetch_and_classify_emails → extract_and_store_job_postings → create_calendar_event
```

해결 전략

1. **이메일 자동 분류** — Gmail/Daum IMAP에서 최근 메일을 수집하여 7개 카테고리로 AI 분류
2. **채용공고 구조화 저장** — 이메일 본문·HTML 링크에서 직무명·회사명·URL·마감일 추출 → 벡터 DB + SQLite FTS5 이중 저장
3. **이력서 분석 및 매칭** — PDF 이력서를 업로드하면 AI가 기술 스택·강점·적합 직무를 분석하고 저장된 채용공고와 유사도 매칭
4. **캘린더 자연어 제어** — "5월 3일 오후 2시 면접" 같은 한국어 명령을 Google Calendar에 자동 등록
5. **슬랙 알림** — 중요 정보를 Slack 웹훅으로 즉시 전송

---

3. 트레이드오프 및 설계 결정

3-1. Single Agent vs Multi-Agent

항목	Single Agent (채택)	Multi-Agent
구현 복잡도	낮음 — 하나의 ReAct 루프	높음 — 오케스트레이터 + 서브에이전트 설계 필요
도구 간 컨텍스트 공유	동일 세션 딕셔너리로 자연스럽게 공유	에이전트 간 메시지 직렬화 필요
확장성	도구 추가로 기능 확장 용이	전문화된 에이전트 분업 가능
적합한 규모	도메인이 명확하고 순차 작업 중심인 현재 요구사항	병렬 처리·대규모 워크플로우 필요 시

결정: 현재 요구사항(이메일→채용공고→캘린더의 선형 파이프라인)은 Single Agent로 충분히 커버되고, 개발·디버깅 비용이 낮으므로 Single Agent를 채택.

3-2. 벡터 DB + 키워드 DB 하이브리드 저장

저장소	역할	이유
ChromaDB (벡터)	의미 기반 유사도 검색	"Python 백엔드 개발자" → 표현이 다른 공고도 검색
SQLite FTS5 (키워드)	정확한 키워드 매칭	회사명·직무명 정확 검색, 오프라인 가능

최종 점수 = 벡터 점수 × 0.6 + 키워드 점수 × 0.4 로 랭킹.

3-3. 인증 방식 — JWT Bearer Token

FastAPI 백엔드는 JWT로 모든 엔드포인트를 보호한다. Streamlit 세션에 토큰을 저장하여 프론트-백엔드 간 인증을 유지한다.

- 장점: 상태 비저장(stateless), Docker 환경에서 세션 공유 불필요
- 단점: 토큰 만료(24시간) 시 재로그인 필요 → 향후 refresh token으로 개선 가능

3-4. 채용공고 스크래핑 — requests 우선, Playwright 폴백

```
scrape_job_page(url)
  ├─ playwright_auth.json 존재 시: Playwright (JS 렌더링 지원)
  └─ 기본: requests + BeautifulSoup (속도 우선)
```

Playwright는 uvicorn의 이벤트 루프와 충돌하므로 `ThreadPoolExecutor` + `asyncio.run()` 패턴으로 별도 스레드에서 실행.

3-5. 날짜 해석 — KST 현재 날짜 주입

LLM이 "5월 3일"처럼 연도 없는 날짜를 과거 연도로 해석하는 문제를 방지하기 위해, 시스템 프롬프트 맨 앞에 오늘 날짜(KST)를 매 요청마다 주입한다.

---

4. 시스템 아키텍처

```
┌──────────────────────────────────────────────────────────┐
│                    Streamlit Frontend                    │
│                                                          │
│  ┌──────────────┐  ┌────────────────┐  ┌──────────┐     │
│  │ Scheduly Home│  │ Email Category  │  │  Resume   │    │
│  │ 캘린더 + 채팅 │  │ 이메일 분류·추출 │  │  Summary  │    │
│  └──────┬───────┘  └────────┬───────┘  └────┬─────┘     │
│         └──────────────────┼──────────────────┘         │
│                    utils.py (requests)                   │
└──────────────────────────┬───────────────────────────────┘
                           │
                           │ HTTP / JWT
┌──────────────────────────▼───────────────────────────────┐
│                    FastAPI Backend                       │
│                                                          │
│   POST /chat ──► agent.py (ReAct Loop, gpt-4o)          │
│   GET  /calendar/month ──► calendar_tools                │
│   POST /auth/login     ──► JWT 발급                      │
│                                                          │
│   ┌──────────────────────────────────────────────┐       │
│   │                   Tools                      │       │
│   │  email_tools │ calendar_tools │ resume_tools  │       │
│   │  slack_tools │ search_tools                  │       │
│   └──────┬──────────────────────┬──────────────┘       │
│          │                      │                        │
│   ┌──────▼──────┐     ┌──────────▼──────────┐           │
│   │  core/      │     │  External APIs      │           │
│   │  indexing   │     │  OpenAI (gpt-4o)    │           │
│   │  retrieval  │     │  Gmail API          │           │
│   │  vector_store     │  Google Calendar API │           │
│   │  keyword_store    │  Slack Webhook      │           │
│   │  google_auth│     │  Tavily Search      │           │
│   └─────────────┘     └────────────────────┘           │
│                                                          │
│   ChromaDB (벡터)  SQLite FTS5 (키워드)                  │
└──────────────────────────────────────────────────────────┘
```

디렉토리 구조

```
08_SingleAgent/
├── backend/
│   ├── main.py                 # FastAPI 앱, JWT 인증, 라우터
│   ├── agent.py                # ReAct 루프 (OpenAI tool-calling)
│   ├── prompts/
│   │   └── system_prompt.txt   # 에이전트 시스템 프롬프트
│   ├── tools/
│   │   ├── __init__.py         # Tool 정의 + 실행 라우터
│   │   ├── email_tools.py      # 이메일 fetch·분류·채용공고 추출
│   │   ├── calendar_tools.py   # Google Calendar CRUD
│   │   ├── resume_tools.py     # 이력서 분석·채용공고 매칭
│   │   ├── search_tools.py     # RAG 검색·웹 검색
│   │   └── slack_tools.py      # Slack 메시지 전송
│   ├── core/
│   │   ├── schemas.py          # Pydantic 도메인 모델
│   │   ├── indexing.py         # 이메일 파싱·분류·채용공고 구조화
│   │   ├── retrieval.py        # 스크래핑·요약·이력서 분석
│   │   ├── vector_store.py     # ChromaDB 저장·검색
│   │   ├── keyword_store.py    # SQLite FTS5 저장·검색
│   │   ├── google_auth.py      # Gmail·Calendar OAuth2
│   │   └── search.py           # 하이브리드 검색 유틸
│   ├── requirements.txt
│   └── Dockerfile
├── frontend/
│   ├── streamlit_app.py        # 앱 진입점, 로그인·네비게이션
│   ├── home.py                 # Scheduly Home (채팅 + 캘린더)
│   ├── email_category.py       # Email Category 페이지
│   ├── resume_summary.py       # Resume Summary 페이지
│   ├── utils.py                # 공통 API 호출 헬퍼
│   └── requirements.txt
└── docker-compose.yml
```

---

5. 에이전트 파이프라인

ReAct 루프 (agent.py)

```
사용자 메시지
    │
    ▼
┌──────────────────────────────┐
│  시스템 프롬프트 구성          │
│  "오늘 날짜: 2026-04-28 (화)"  │
│  + system_prompt.txt         │
└──────────────┬───────────────┘
               │
    ┌──────────▼──────────────────────────┐
    │         OpenAI gpt-4o               │  ◄── MAX_ITERATIONS = 10
    │         tool_choice = "auto"        │
    └──────────┬──────────────────────────┘
               │
       finish_reason?
          ┌────┴────┐
     tool_calls    stop
         │          │
         ▼          ▼
    execute_tool   최종 응답 반환
    (tools/)
         │
    tool 결과를
    messages에 추가
         │
    다음 이터레이션 ◄──────────────┘
```

12개 도구 목록

도구	기능	연동 서비스
`fetch_and_classify_emails`	이메일 수집 및 7개 카테고리 분류	Gmail API / Daum IMAP
`summarize_business_emails`	업무 이메일 상세 요약	OpenAI gpt-4o-mini
`extract_and_store_job_postings`	채용공고 추출 → 스크래핑 → DB 저장	OpenAI + ChromaDB + SQLite
`search_jobs`	채용공고 하이브리드 검색	ChromaDB + SQLite FTS5
`analyze_resume`	이력서 분석 및 저장	OpenAI gpt-4o-mini
`list_calendar_events`	특정 날짜 일정 조회	Google Calendar API
`create_calendar_event`	일정 등록 (시간 지정 / 종일)	Google Calendar API
`delete_calendar_event`	일정 삭제	Google Calendar API
`move_calendar_event`	일정 날짜 이동	Google Calendar API
`send_slack_message`	Slack 메시지 전송	Slack Webhook
`rag_search`	내부 벡터 DB 의미 검색	ChromaDB
`web_search`	인터넷 검색	Tavily API

세션 컨텍스트 흐름

```
세션 시작 (session_id 생성)
    |
fetch_and_classify_emails 실행
    |  context["_emails"] = [...]
    |  context["_business_emails"] = [...]
    |  context["_newsletter_emails"] = [...]
    |  context["_classifications"] = [...]
    |
extract_and_store_job_postings 실행
    |  (context에서 emails 읽어 채용공고 추출)
    |
search_jobs / analyze_resume
    |  (DB에서 조회)
    |
create_calendar_event 실행
    |  (추출된 마감일·면접 날짜를 캘린더에 등록)
    ▼
응답 반환
```

이메일 fetch → 채용공고 추출 → 캘린더 등록까지 **하나의 세션** 안에서 컨텍스트가 이어진다.

---

6. 프로젝트 진화 경로

1단계 — 기본 이메일 분류기 (07_Advanced_classifier)

- Gmail 이메일 수집 + OpenAI 분류
- 단순 Streamlit 단일 페이지 UI
- 결과 표시만 가능, 액션 없음

2단계 — Single Agent 도입 (08_SingleAgent v1)

- FastAPI 백엔드 + Streamlit 프론트엔드 분리
- JWT 인증 추가
- ReAct 루프 기반 에이전트 구현
- Google Calendar 연동

3단계 — 멀티 페이지 UI + UX 개선

개선 항목	변경 내용
UI 구조	단일 페이지 → 3-페이지 사이드바 네비게이션
홈 화면	채팅 입력 + 월별 캘린더 나란히 표시
입력 UX	Enter 키 제출, 입력창 자동 클리어, 스크롤 가능한 대화 기록
날짜 해석	KST 현재 날짜를 시스템 프롬프트에 주입 → 연도 오해석 수정
이력서 분석	PDF 업로드 → 분석 → 채용공고 매칭 페이지 추가

4단계 — 안정성 강화

버그	수정 내용
채용공고 추출 500 에러	빈 이메일 조기 반환을 import 앞으로 이동 + 전체 try/except 추가
Playwright asyncio 충돌	`ProactorEventLoop` → `ThreadPoolExecutor` + `asyncio.run()` 패턴으로 교체
ChromaDB 저장 실패 전파	`store_job_postings` try/except 추가로 저장 실패가 500을 유발하지 않도록 처리

---

7. 기술 스택

Backend

분류	기술	용도
웹 프레임워크	**FastAPI**	REST API 서버
AI 모델	**OpenAI GPT-4o**	에이전트 추론, 이메일 분류, 채용공고 추출
AI 모델 (경량)	**OpenAI GPT-4o-mini**	이메일 요약, 이력서 분석
인증	**JWT (python-jose)**	Bearer Token 기반 API 보호
벡터 DB	**ChromaDB**	채용공고·이력서 의미 기반 검색
키워드 DB	**SQLite FTS5**	채용공고·이력서 정확 키워드 검색
임베딩	**text-embedding-3-small**	벡터 생성 (ChromaDB 연동)
이메일	**Gmail API / IMAP**	Gmail 및 Daum 메일 수집
캘린더	**Google Calendar API**	일정 CRUD
웹 스크래핑	**requests + BeautifulSoup**	채용공고 페이지 수집
고급 스크래핑	**Playwright**	JS 렌더링 필요 페이지 (선택)
웹 검색	**Tavily API**	회사 정보·채용 트렌드 검색
알림	**Slack Webhook**	중요 정보 슬랙 전송
배포	**Docker + docker-compose**	컨테이너 배포

Frontend

분류	기술	용도
UI 프레임워크	**Streamlit**	멀티 페이지 웹 앱
캘린더 컴포넌트	**streamlit-calendar**	FullCalendar 기반 월별 캘린더
PDF 파싱	**pdfplumber / PyMuPDF**	이력서 PDF 텍스트 추출
HTTP 클라이언트	**requests**	FastAPI 백엔드 호출

데이터 모델 (Pydantic)

```
Email          — id, subject, sender, date, body, html_body
Classification — index, category, summary
BusinessSummary — index, subject, key_points, action_required, detail_summary
JobPosting     — job_title, company, location, source_email, url, deadline
JobPostingResult — JobPosting + sections[]
ResumeAnalysis — name, suitable_jobs, skills, characteristics, strengths, job_keywords
```

---

8. 실행 방법

사전 준비

```
1. Google Cloud Console에서 Gmail API + Calendar API 활성화
2. OAuth2 credentials.json 다운로드 → backend/ 에 위치
3. OpenAI API 키 발급
4. (선택) Tavily API 키, Slack Webhook URL 발급
```

환경 변수 설정

```
# backend/.env
OPENAI_API_KEY=sk-...
JWT_SECRET_KEY=your-secret-key
ADMIN_USERNAME=admin
ADMIN_PASSWORD=your-password
TAVILY_API_KEY=tvly-...          # 선택
SLACK_WEBHOOK_URL=https://hooks.slack.com/...  # 선택
```

```
# frontend/.streamlit/secrets.toml
BACKEND_URL = "http://localhost:8000"
```

로컬 실행

```
# 백엔드 (터미널 1)
cd 08_SingleAgent/backend
pip install -r requirements.txt
uvicorn main:app --reload --port 8000

# Google OAuth 토큰 최초 발급 (브라우저 팝업)
python core/google_auth.py

# 프론트엔드 (터미널 2)
cd 08_SingleAgent/frontend
pip install -r requirements.txt
streamlit run streamlit_app.py
```

접속 URL: http://localhost:8501
기본 계정: `admin` / `admin1234`

Docker 실행

```
cd 08_SingleAgent
docker-compose up --build
```

백엔드: http://localhost:8000
Swagger UI: http://localhost:8000/docs

주요 사용 시나리오

이메일 분류 및 채용공고 추출

```
Email Category 페이지
  → [이메일 가져오기 & 분류] 클릭
  → 분류 결과 확인 (업무/비즈니스, 뉴스레터 등)
  → [채용공고 추출] 클릭
  → 채용공고 목록 저장 완료
```

이력서 매칭

```
Resume Summary 페이지
  → PDF 이력서 업로드
  → [이력서 분석] 클릭
  → 적합 직무·기술 스택 확인
  → [저장된 채용공고와 매칭] 클릭
  → 유사도 순위 결과 확인
```

캘린더 자연어 제어

```
Scheduly Home 페이지 입력창
  → "5월 10일 오후 2시 A회사 면접 등록해줘"
  → Google Calendar 자동 등록 완료
  → 우측 캘린더에서 즉시 확인
```

일정 이동

```
입력창: "5월 10일 A회사 면접을 5월 15일로 옮겨줘"
  → 에이전트: list_calendar_events → move_calendar_event
  → 시간은 유지한 채 날짜만 변경
```